Guion de exposición

Colisiones en videojuegos · Estructura de Datos II (IST 4031) · Universidad del Norte · 2026-30

Cada ficha corresponde a una diapositiva de `01_Presentacion_Colisiones_Godot.pdf`, en el mismo orden. El tiempo se estima a unas 130 palabras por minuto, e incluye 2 min 30 s de ejecución en vivo en la diapositiva 18; el tiempo asignado es de 25 minutos más 5 de preguntas.

Integrante	Diapositivas	Tiempo estimado
María Yusti	1 a 5 y 19	6:00
Axel Tornay	6 a 9	5:30
Juan Vizcaíno	10 a 15	5:45
Isac Monterrosa	16 a 18	6:15
Total		**23:30**

DIAPOSITIVA 1 · RESPONSABLE: MARÍA YUSTI · 0:15

Portada

Buenos días, profesor y compañeros. Hoy vamos a explicar cómo un videojuego decide cuándo dos objetos se tocan y cómo logra hacerlo rápido usando estructuras de datos. Somos María Yusti, Axel Tornay, Juan Vizcaíno e Isac Monterrosa. Además de la teoría, mostraremos una implementación práctica en Godot.

Transición: Empecemos por entender el problema.

DIAPOSITIVA 2 · RESPONSABLE: MARÍA YUSTI · 0:30

Índice

El recorrido tiene ocho partes. Primero, la introducción y los fundamentos. Segundo, las formas de colisión. Tercero, la detección exacta o fase estrecha. Cuarto, la detección discreta y continua. Quinto, cómo escalar con estructuras de datos. Sexto, aplicaciones reales. Séptimo, la implementación en Godot. Y octavo, conclusiones y referencias.

Transición: Vamos con la introducción.

DIAPOSITIVA 3 · RESPONSABLE: MARÍA YUSTI · 1:45

Qué es una colisión y por qué es un problema

Un videojuego avanza por cuadros, y en cada cuadro debe decidir qué objetos se tocan y qué hacer con ese contacto. Eso parece sencillo, pero es difícil por dos razones. A 60 cuadros por segundo hay unos 16,7 milisegundos para todo el trabajo, y con n objetos existen n por n menos uno, dividido entre dos, pares posibles. Aquí está la relación con Estructuras de Datos: ordenar, usar hash, árboles y grafos decide cuántas pruebas hacemos; es el mismo problema de buscar sin revisar todo. Abajo vemos el origen del problema: el tiempo se simula en pasos discretos. En el cuadro tres la pelota ya está junto a la pared, pero solo conocemos posiciones por cuadro. De ahí nacen la detección, el túnel y el costo. Piensen en un juego de plataformas: el personaje camina, salta y en cada cuadro el juego debe responder si sus pies tocan el suelo, si chocó con un enemigo o si recogió una moneda. Si hay pocos objetos, revisar todo contra todo funciona. Pero un juego con cientos de objetos necesita una forma más inteligente. Por eso hablamos de este tema en Estructuras de Datos: es el mismo problema de organizar la información para no revisar todo cada vez, que es justo lo que hacen los árboles, el hash y el ordenamiento.

Transición: Antes de seguir, fijemos algunas definiciones.

DIAPOSITIVA 4 · RESPONSABLE: MARÍA YUSTI · 1:30

Definiciones necesarias

Estas son las definiciones que vamos a usar. Un colisionador es una forma simple que representa al objeto. La AABB es una caja alineada con los ejes, la forma más barata. El contacto es el resultado de una prueba: una normal, que da la dirección, y una profundidad, que dice cuánto se solapan. La fase amplia descarta rápido los pares que no pueden chocar, y la fase estrecha prueba con exactitud los candidatos. La detección discreta revisa solo las posiciones de cada cuadro; la continua revisa el recorrido entre cuadros. Y el túnel ocurre cuando un objeto rápido cruza a otro sin solaparse en ningún cuadro. Quiero resaltar dos definiciones que se confunden con frecuencia. La primera es contacto: no basta con saber que hay colisión, también necesitamos la normal, que indica en qué dirección separar los objetos, y la profundidad, que indica cuánto. La segunda es la diferencia entre fase amplia y fase estrecha: la amplia es un filtro barato que puede equivocarse dejando pasar pares que no chocan, pero no puede descartar uno que sí choca; la estrecha confirma con exactitud.

Transición: Veamos cómo encajan estas piezas en un proceso.

DIAPOSITIVA 5 · RESPONSABLE: MARÍA YUSTI · 1:00

El proceso: de la lista de objetos al contacto

El proceso tiene tres etapas. La fase amplia pregunta quién podría chocar y usa cajas, cuadrículas o árboles. La fase estrecha pregunta si chocan de verdad y cómo, con distancias, SAT o proyecciones. Y la respuesta decide qué hace el juego: rebotar, deslizar o emitir una señal. Fíjense en la diferencia entre detectar y responder: una moneda detecta que entró el jugador y no bloquea nada, mientras una pelota responde rebotando. Todo esto se repite en cada cuadro, unas 60 veces por segundo, y cada etapa descarta trabajo para la siguiente. Un ejemplo con el proyecto que veremos: la moneda es un Area2D, así que solo detecta. La pelota es un RigidBody2D, así que además responde. Es la misma pregunta, ¿se tocan?, con respuestas distintas. Entender esta separación es clave porque cada etapa se puede optimizar y programar por separado.

Transición: Ahora Axel explica con qué formas se representan los objetos.

DIAPOSITIVA 6 · RESPONSABLE: AXEL TORNAY · 1:30

Del modelo detallado al colisionador

Los modelos reales son complejos, así que los juegos los representan con formas simples. La AABB necesita pocas comparaciones y es ideal como filtro, pero es imprecisa si el objeto rota. El círculo o la esfera necesita una distancia y un radio, y no depende de la rotación, aunque ajusta mal a objetos alargados. La caja orientada sigue la rotación, pero es más cara. La cápsula es buena para personajes. El polígono o casco convexo se ajusta mejor al contorno, pero su costo crece con las aristas. Y la malla de triángulos es exacta pero muy costosa, por eso se reserva para objetos estáticos. Además, un personaje puede dividirse en varios hitboxes, como cabeza, torso y extremidades, para saber dónde recibe el impacto, como muestra el video de referencia. La regla práctica de esta tabla es que cuanto más simple la forma, más barata es la prueba, pero menos exacta. Por eso casi nunca se usa la malla completa para todo: se elige una forma que equilibre precisión y costo según lo que el objeto necesita. Un personaje de plataformas puede conformarse con una caja; un escenario complejo puede usar mallas, pero solo porque no se mueve.

Transición: Veamos cómo se prueba la colisión con las formas más simples.

DIAPOSITIVA 7 · RESPONSABLE: AXEL TORNAY · 1:30

Fase estrecha: las pruebas simples

Empecemos con dos círculos de radio 10 cuyos centros están a distancia 15. Calculamos el vector entre centros y preguntamos si su longitud es menor que la suma de radios: 15 es menor que 20, hay colisión y la profundidad es 5. A la derecha, dos cajas. Proyectamos en X y en Y, y se solapan 5 en X y 15 en Y. Como hay solape en ambos ejes, chocan, y el menor solape, el del eje X, da la normal. Ambas pruebas cuestan tiempo constante y espacio constante. Su ventaja es que son exactas y baratas; su limitación es que solo cubren círculos y cajas alineadas. Veamos la primera prueba con calma. Elevamos al cuadrado la distancia para evitar calcular una raíz cuadrada, algo que es más costoso. La profundidad de 5 significa que cada círculo se metió 2,5 en el otro; en la práctica, ese valor se usa para separarlos. En la prueba de cajas, que el menor solape sea el del eje X nos dice que lo más barato para separarlas es empujarlas en horizontal. Como ambas pruebas son pocas operaciones aritméticas, su costo no depende de cuántos objetos haya.

Transición: Para polígonos necesitamos otra herramienta.

DIAPOSITIVA 8 · RESPONSABLE: AXEL TORNAY · 1:15

SAT: el teorema del eje separador

Para polígonos convexos se usa el teorema del eje separador. Uno: elegimos como eje la normal de una arista. Dos: proyectamos las dos formas sobre ese eje y obtenemos dos segmentos. Tres: miramos si se solapan; en este ejemplo el solape es 3,54. Cuatro: si en algún eje hay un hueco, no chocan y paramos. Si ningún eje tiene hueco hay colisión, y el menor solape da el vector de traslación mínima. El costo crece con el cuadrado del número de aristas. Su limitación es que solo sirve para formas convexas; las cóncavas se parten en convexas. Piénsenlo como proyectar sombras: si dos objetos tienen sombras separadas sobre alguna pared, es imposible que estén tocándose. SAT busca esa pared. En la diapositiva, el cuarto cuadro muestra un eje donde hay un hueco verde entre las proyecciones, y con eso basta para descartar la colisión sin revisar el resto de ejes. Esa salida temprana es lo que hace eficiente al algoritmo en la mayoría de casos.

Transición: Hasta aquí revisamos posiciones fijas; ¿qué pasa si el objeto es muy rápido?

DIAPOSITIVA 9 · RESPONSABLE: AXEL TORNAY · 1:15

Cuándo mirar: en cada cuadro o a lo largo del trayecto

La detección discreta compara solo las posiciones de cada cuadro. Es barata y simple, pero puede producir túnel: la pelota está antes de la pared en un cuadro y después en el siguiente, y nunca se solapan. La detección continua revisa el trayecto y calcula el instante del impacto. Por ejemplo, si la pared está a 2 metros y la bala va a 2 metros por segundo, el impacto ocurre en 1 segundo, que es la distancia dividida entre la velocidad. No atraviesa, pero es más costosa, por eso se reserva para balas o vehículos. Otras salidas son dividir el paso de física, usar paredes más gruesas o lanzar un rayo. En Godot, el RigidBody2D tiene la propiedad continuous_cd, desactivada por defecto. Este tema es importante porque aparece incluso en juegos modernos. Un personaje que camina rara vez produce túnel, pero un proyectil sí. La detección continua cuesta más, por eso se activa solo donde hace falta. Es una decisión de ingeniería: pagar más por los objetos críticos y menos por el resto.

Transición: Ahora Juan explica qué pasa cuando hay muchos objetos.

DIAPOSITIVA 10 · RESPONSABLE: JUAN VIZCAÍNO · 0:30

El problema: el costo crece con el cuadrado

Con cinco objetos hay diez pares posibles. La fórmula es n por n menos uno, entre dos. Con diez objetos son 45 pares, con cien son 4.950 y con mil son 499.500 en cada cuadro. Eso es O de n al cuadrado. Y a sesenta cuadros por segundo tenemos unos 16,7 milisegundos, así que probar con exactitud todos los pares no cabe. Esto conecta con lo visto de complejidad: necesitamos reducir los pares antes de la fase estrecha.

Transición: La solución es una fase amplia.

DIAPOSITIVA 11 · RESPONSABLE: JUAN VIZCAÍNO · 1:15

Fase amplia: evitar probar todos los pares

Hay tres ideas. Ordenar por un eje, o sweep and prune: solo comparamos los objetos que se solapan en X; cuesta n log n más k, donde k son los pares que de verdad se solapan. La cuadrícula con hash: cada objeto cae en una celda según su posición y solo comparten pares los de la misma celda; cuesta n más k. Y el QuadTree: cada región se divide en cuatro donde hay más objetos, con costo aproximado de n log n. En los tres casos, k es el número de pares que realmente se solapan. En el sweep and prune cada objeto ocupa un intervalo sobre el eje X; ordenamos los intervalos y solo comparamos los que se traslapan. En la cuadrícula, la fórmula de la celda toma la posición, la divide entre el tamaño de celda y toma la parte entera. En el QuadTree, la subdivisión solo ocurre donde hay muchos objetos, así que las zonas vacías no cuestan nada. Después de la fase amplia, la fase estrecha solo revisa los pares que sobrevivieron.

Transición: Estas ideas ya las vimos en clase.

DIAPOSITIVA 12 · RESPONSABLE: JUAN VIZCAÍNO · 1:15

Lo que vimos en clase, aplicado

Cada técnica corresponde a un tema del curso. En árboles, la altura determina la eficiencia, y el QuadTree divide el espacio en cuatro. En ordenamientos, ordenar sirve para comparar menos, como en sweep and prune. En tablas hash, la cuadrícula convierte la posición en una celda, como una clave que da una posición. En grafos, los cuerpos son vértices, los contactos son aristas y los grupos son componentes conexas. Y en hilos, la idea de repartir un problema grande, aunque no la implementamos. Esta es la conexión más importante con el curso. Lo que vimos de árboles y su altura se aplica directamente: un QuadTree desbalanceado pierde eficiencia, igual que un árbol binario degenerado. Lo de hash también aplica, pero con cuidado: una colisión de hash, dos claves en la misma posición, no es lo mismo que una colisión física. Y en el proyecto sí implementamos el recorrido DFS sobre un grafo de contactos para contar grupos de cuerpos que se tocan.

Transición: Comparemos todas las técnicas.

DIAPOSITIVA 13 · RESPONSABLE: JUAN VIZCAÍNO · 0:45

Comparación: ventajas y limitaciones

Esta tabla resume ventajas y limitaciones. AABB y círculo son muy baratas pero aproximadas, y encajan en plataformas 2D. SAT es exacta pero no sirve para cóncavos. Sweep and prune aprovecha que los objetos se mueven poco entre cuadros, pero se degrada si muchos se solapan en el eje. La cuadrícula consulta casi en tiempo constante, pero sufre con tamaños muy distintos. El QuadTree se adapta a zonas densas y vacías, pero hay que actualizarlo si los objetos se mueven. La continua evita el túnel pero es más cara. La conclusión es que no hay técnica universal: los motores combinan una fase amplia barata con una estrecha exacta.

Transición: Veamos dónde se usan.

DIAPOSITIVA 14 · RESPONSABLE: JUAN VIZCAÍNO · 1:15

Cuatro aplicaciones reales de las técnicas

Primera aplicación, plataformas 2D: la caja del personaje se prueba solo contra las baldosas vecinas de una cuadrícula, y la respuesta es frenar o deslizar. Segunda, disparos en 3D: el personaje tiene varias cajas por parte del cuerpo y las balas rápidas usan rayos o detección continua para registrar el impacto y la zona herida. Tercera, mundos de bloques: la caja de cada entidad se compara solo con los bloques que ocupa. Cuarta, billar y física: círculos o esferas y una respuesta por rebote según la normal del contacto. Estos cuatro ejemplos usan las mismas ideas con distinta combinación. En plataformas la cuadrícula es natural porque el mapa ya está hecho de baldosas. En disparos, lo crítico es la precisión del impacto y por eso se usan varias cajas y rayos. En un mundo de bloques, la cuadrícula entera hace que la consulta sea muy barata. Y en billar, la fase estrecha de círculos es suficiente porque son pocos objetos y la respuesta física es lo importante.

Transición: Ahora pasamos a Godot, que es donde aplicamos todo esto.

DIAPOSITIVA 15 · RESPONSABLE: JUAN VIZCAÍNO · 0:45

Cuatro tipos de cuerpo

Godot ofrece cuatro tipos de cuerpo. El Area2D detecta y avisa con señales sin bloquear; en el proyecto es la moneda. El StaticBody2D no se mueve; son las paredes y el obstáculo. El RigidBody2D deja que el motor calcule la respuesta; es la pelota. Y el CharacterBody2D es aquel donde programamos el movimiento y la respuesta; es el jugador. Cada uno lleva un CollisionShape2D con su forma. Esto importa porque la elección decide quién calcula la respuesta: el motor o el programador.

Transición: Isac continúa con las capas y el código.

DIAPOSITIVA 16 · RESPONSABLE: ISAC MONTERROSA · 1:15

Capas y máscaras filtran quién choca con quién

collision_layer dice en qué capa está el objeto, y collision_mask qué capas escanea. En el proyecto hay cuatro capas: paredes, jugador, pelotas y monedas. El jugador choca con paredes y pelotas; la pelota con paredes, jugador y otras pelotas; y la moneda solo escanea al jugador. La ventaja es que filtramos pares sin escribir código; la limitación es que hay 32 capas y, si la máscara está mal, el objeto simplemente no choca ni avisa. Por ejemplo, con la máscara de la moneda en la capa 2, las tres pelotas pasan sobre ella sin activar la señal body_entered. Leamos la tabla: cada fila es un objeto y cada casilla marcada es una capa que ese objeto escanea. Como las capas se guardan como bits, la pelota en la capa tres se guarda con el valor 4, y el jugador, que escanea las capas uno y tres, guarda 5. No es necesario memorizar esto, porque en el editor se configuran con nombres, pero explica los números que se ven en los archivos de escena.

Transición: Veamos el código y la escena.

DIAPOSITIVA 17 · RESPONSABLE: ISAC MONTERROSA · 1:15

Escena y código del proyecto

A la izquierda está la escena laboratorio, con paredes, obstáculo, jugador, tres pelotas, tres monedas, una capa de superposición y la interfaz. A la derecha, dos formas de reaccionar. En el jugador, después de move_and_slide, revisamos si hubo colisiones, y emitimos una señal con el objeto y la normal: la respuesta la programamos nosotros. En la moneda solo detectamos: cuando entra un cuerpo del grupo jugador, emitimos la señal y la moneda se elimina. El rebote de la pelota lo calcula el motor. Del lado del código, vale la pena notar la diferencia de responsabilidad. En el jugador, nosotros leemos la normal del contacto y decidimos qué hacer: este es el paso de respuesta programada. En la moneda no hay física: solo una señal. Y la pelota no tiene código de rebote: esa respuesta la calcula el motor con el material físico de rebote uno y fricción cero.

Transición: Pasemos a la demostración.

DIAPOSITIVA 18 · RESPONSABLE: ISAC MONTERROSA · 3:45

Demostración y origen del código

Vamos a ejecutar cinco casos. Uno, mover al jugador contra una pared: se detiene y se desliza, y sube el contador de choques. Dos, dejar que una pelota golpee al jugador: rebota y sube el contador de rebotes. Tres, tocar una moneda: desaparece y sube el contador. Cuatro, pulsar Espacio: se dibujan las cajas AABB y las líneas de contacto. Cinco, ejecutar las pruebas manuales: debe imprimir 17 pruebas y cero fallos. Sobre el origen del código: lo escribió el grupo con asistencia de inteligencia artificial, a partir de Ericson, de la teoría SAT y de la documentación oficial de Godot. Además de la demostración, quiero mencionar un detalle de honestidad académica: el código fue elaborado con apoyo de inteligencia artificial, y por eso lo declaramos en esta diapositiva, junto con las fuentes teóricas y la documentación que lo respaldan. En las pruebas manuales usamos casos calculados a mano, los mismos de las trazas de la fase estrecha: círculos de radio 10 a distancia 15, que dan profundidad 5, y rombos que dan 3,54.

Transición: Cerremos con las conclusiones.

DIAPOSITIVA 19 · RESPONSABLE: MARÍA YUSTI · 1:00

Conclusiones

Cinco ideas. Una: detectar y responder son pasos distintos, y el contacto los conecta. Dos: se filtra primero con formas y estructuras baratas y luego se mide con exactitud. Tres: discreta o continua depende de la velocidad y el tamaño del objeto. Cuatro: las estructuras de datos de la clase reducen el costo de O de n al cuadrado. Cinco: no hay técnica universal; cada juego equilibra precisión y rendimiento. Muchas gracias; quedamos atentos a sus preguntas. Si tuviéramos que quedarnos con un mensaje, sería este: una colisión se resuelve en dos preguntas, primero quién podría chocar y luego si de verdad chocan, y las estructuras de datos de la clase son las que hacen barata la primera.

Transición: Las referencias quedan proyectadas.

DIAPOSITIVA 20 · RESPONSABLE: TODOS (NO SE LEE) · —

Referencias

Queda proyectada durante las preguntas, en formato IEEE.